Plan of the chapter



1. Introduction
2. SQL data types
3. Common SQL Commands
4. SQL operators
5. Database and relational database design
6. Database normalization

- The normalization process aims to:
  - minimize data duplications,
  - avoid errors during data modifications,
  - and simplify data queries from the database.

- The three fundamental normalization forms are known as:

- **First Normal Form (1NF)**
- **Second Normal Form (2NF)**
- **Third Normal Form (3NF)**

❖ **Example:**

The table includes fictitious data required by a Medical Group Surgery based in London to generate relevant reports.

- Doctors work in multiple regions and various councils in London.
- Once a patient books an appointment, they are given a slot ID at their local surgery.
- There might be multiple surgeries in the same council but with different postcodes, where one or more councils belong to a particular region, for example, East or West London.

Doctor ID	Doctor name	Region	Patient ID	Patient name	Surgery Number	Surgery council	Postcode	Slot ID	Total Cost
D1	Karl	West London	P1 P2 P3	Rami Kim Nora	3	Harrow	HA9SDE	A1 A2 A3	1500 1200 1600
D1	Karl	East London	P4 P5	Kamel Sami	4	Hackney	E1 6AW	A1 A2	2500 1000
D2	Mark	East London	P5 P6	Sami Norma	4	Hackney	E1 6AW	A3 A4	1500 2000
D2	Mark	West London	P7 P1	Rose Rami	5	Harrow	HA862E	A4 A5	1000 1500

❖ **Example:**

- The data listed in the table are in an **unnormalized form**: Repeating groups of data appear in many cases, for instance, doctors, regions and council names.
- There are also multiple instances of data stored in the same cell such as with the patient name and total cost columns.

➢ This makes it difficult to update and query data.

➢ It is not easy to choose a unique key and assign it as a primary key.

- Creating this unnormalized table is possible :

➢ **SQL syntax :**

```
CREATE TABLE Surgery( DoctorID VARCHAR(10), DoctorName VARCHAR(50), Region VARCHAR(20), PatientID VARCHAR(10), PatientName VARCHAR(50), SurgeryNumber INT, Council VARCHAR(20), Postcode VARCHAR(10), SlotID VARCHAR(5), TotalCost Decimal);
```

First normal form

- To simplify the data structure of the surgery table, let's apply the **first normal form rules** to enforce the **data atomicity rule** and **eliminate unnecessary repeating groups of data**.
- The data atomicity rule means that you can only have one single instance value of the column attribute in any cell of the table.
- The atomicity problem only exists in the columns of data related to the patients.
- It is important to create a new table for patient data to fix this.
- You can organize all data related to the patient entity in one separate table, where each cell of any column contains only one single instance of data.

First normal form

Patient ID	Patient name	Slot ID	Total Cost
P1	Rami	A1	1500
P2	Kim	A2	1200
P3	Nora	A3	1600
P4	Kamel	A1	2500
P5	Sami	A2	1000
P6	Norma	A5	2000
P7	Rose	A6	1000

- This table includes one single instance of data in each cell, which makes it much simpler to read and understand.
- The patient table requires two columns: the patient ID and the Slot ID together to identify each record in a unique way → You need a composite primary key.

➢ **SQL syntax :**

```
CREATE TABLE Patient(PatientID VARCHAR(10)NOT NULL, PatientName
VARCHAR(50), SlotID VARCHAR(10) NOT NULL, TotalCost Decimal, CONSTRAINT
PK_Patient PRIMARY KEY (PatientID, SlotID));
```

First normal form

Doctor ID	Doctor name	Region	Surgery Number	Surgery council	Postcode
D1	Karl	West London	3	Harrow	HA9SDE
D1	Karl	East London	4	Hackney	E1 6AW
D2	Mark	West London	4	Hackney	E1 6AW
D2	Mark	East London	5	Harrow	HA862E

- You have removed the patient attributes from the main table.
- The table also contains repeating groups of data in each column.

➢ You can fix this by separating the table into two tables of data: the doctor table and the surgery table, where each table deals with one specific entity.

➢ By applying the atomicity rule and removing the repeating groups of data, the database now meets the first normal form.

First normal form

Doctor table

Doctor ID	Doctor name
D1	Karl
D2	Mark

Surgery table

Surgery Number	Region	Surgery council	Postcode
3	West London	Harrow	HA9SDE
4	East London	Hackney	E1 6AW
5	West London	Harrow	HA862E

- In the doctor table, you can identify the doctor ID as a single column primary key.

➢ **SQL syntax :**
```
CREATE TABLE Doctor  (DoctorID VARCHAR(10), DoctorName VARCHAR(50),
PRIMARY KEY (DoctorID));
```

- The surgery table can have the surgery number as a single column primary key.

➢ **SQL syntax :**
```
CREATE TABLE Surgery (SurgeryNumber INT NOT NULL, Region
VARCHAR(20), Council VARCHAR(20), Postcode VARCHAR(10), PRIMARY KEY
(SurgeryNumber));
```

Second normal form

- In the **second normal form**, you need to avoid any **partial dependency** relationships between data.
- Partial dependency refers to tables with a **composite primary key** that consists of a combination of two or more columns, where a non-key attribute value depends only on one part of the composite key.

Patient ID	**Patient name**	**Slot ID**	**Total Cost**
P1	Rami	A1	1500
P2	Kim	A2	1200
P3	Nora	A3	1600
P4	Kamel	A1	2500
P5	Sami	A2	1000
P5	Sami	A3	1000
P6	Sami	A4	1500
P7	Norma	A5	2000
P8	Rose	A6	1000
P1	Rami	A7	1500

©Dr. Kalthoum Zaouali 2026-2027

Second normal form

- In the patient table, you need to check whether there are any non-key attributes depending on one part of the composite key.
- The patient's name is a non-key attribute, and it can be determined by using the patient ID only.
- You can determine the total cost by using the Slot ID only.

➢ Splitting the patient table into two tables: patient table and appointment table.

Patient ID	Patient name
P1	Rami
P2	Kim
P3	Nora
P4	Kamel
P5	Sami
P7	Norma
P8	Rose

➢ **SQL syntax :**

```
CREATE TABLE Patient(PatientID VARCHAR(10) NOT NULL, PatientName VARCHAR(50), PRIMARY KEY (PatientID));
```

Second normal form

- In the appointment table, you need to add a unique key to ensure you have a primary key that can identify each unique record in the table.

Appointment table before adding a new unique key

Slot ID	Total Cost
A1	1500
A2	1200
A3	1600
A1	2500
A2	1000
A3	1000
A4	1500
A5	2000
A6	1000
A7	1500

Appointment table after adding a new unique key

Appointment ID	Slot ID	Total Cost
1	A1	1500
2	A2	1200
3	A3	1600
4	A1	2500
5	A2	1000
6	A3	1000
7	A4	1500
8	A5	2000
9	A6	1000
10	A7	1500

➢ **SQL syntax :**

```
CREATE TABLE Appointments (AppointmentID INT NOT NULL,
SlotID VARCHAR(10), TotalCost Decimal, PRIMARY KEY (AppointmentID));
```

Third normal form

- For a relation in a database to be in the third normal form, it must already be in the second normal form (2NF).
- It must have no transitive dependency: Any non-key attribute in the surgery table may not be functionally dependent on another non-key attribute.
- In the surgery table, the postcode and the council are non-key attributes, and the postcode is dependent on the council.
- If you change the council value, you must also change the postcode. This is called transitive dependency, which is not allowed in the third normal form.

Surgery number	Region	Surgery council	Postcode
3	West London	Harrow	HA9SDE
4	East London	Hackney	E1 6AW
5	West London	Harrow	HA862E

Third normal form

- Changing the value of the council value has a direct impact on the postcode value, because each postcode belongs to a specific council.
- To fix the transitive dependency, you can split this table into two tables: one for the region with the city and one for the surgery.

Location table

Surgery number	Postcode
3	HA9SDE
4	E1 6AW
5	HA862E

➢ **SQL syntax :**

```
CREATE TABLE Location( SurgeryNumber INT NOT NULL, Postcode VARCHAR(10), PRIMARY KEY (SurgeryNumber));
```

Council table

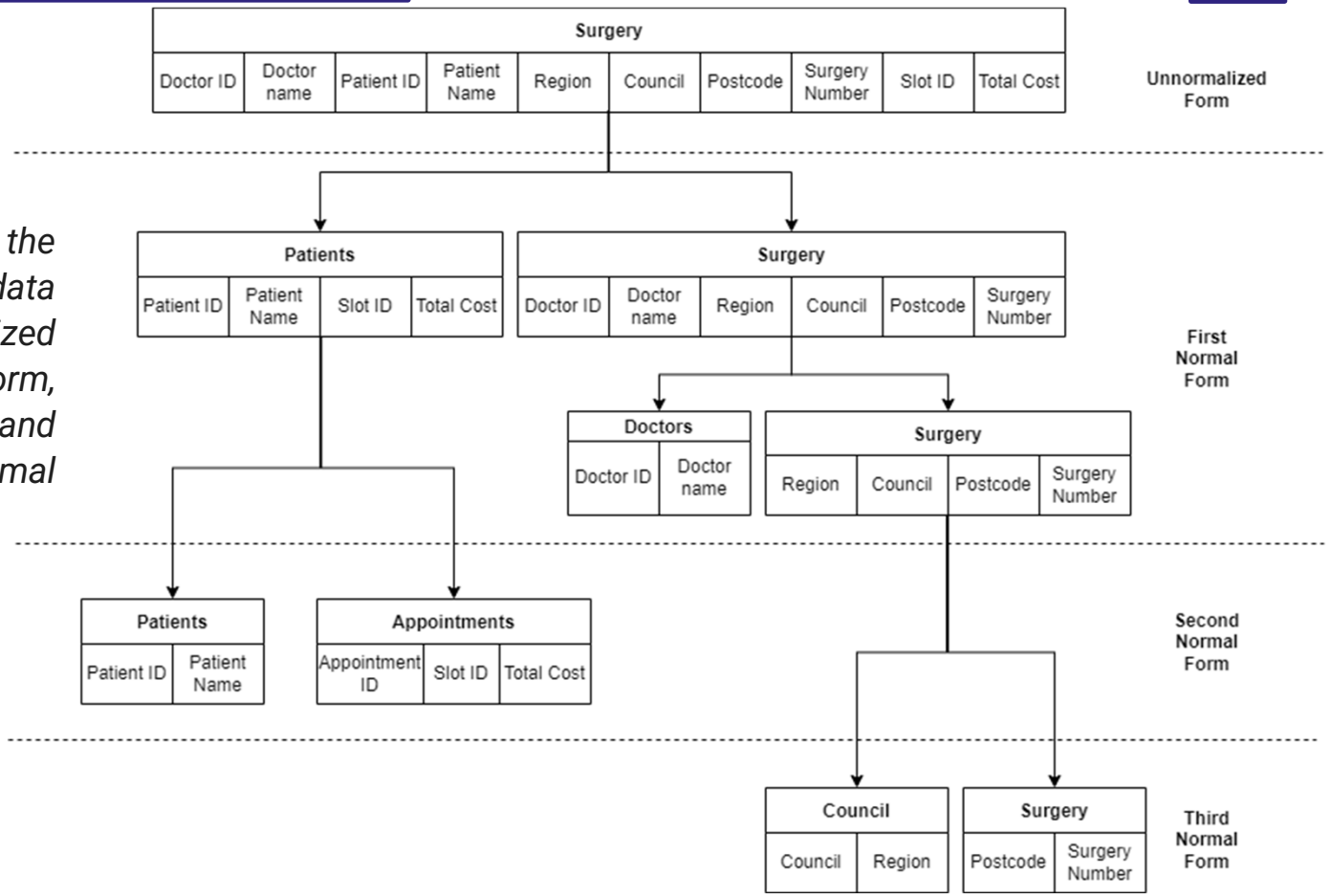
Council	Region
Harrow	West London
Hackney	East London

➢ **SQL syntax :**

```
CREATE TABLE Council (Council VARCHAR(20) NOT NULL, Region VARCHAR(20),PRIMARY KEY (Council));
```

Database normalization

- *This diagram illustrates the stages through which the data moves from the unnormalized form to the first normal form, the second normal form and finally to the third normal form.*



Database normalization

- *It's important to link all tables together to ensure that you have well-organized and related tables in the database. This can be done by defining foreign keys in the tables.*

